

Person 3 -- Interfaces (Caller UI, Agent Console, Admin Dashboard)

NHAA Trauma Triage System -- Standalone Build Specification

How to Use This Document

This is a complete, standalone build specification. Build every phase directly, in order. You never need to wait for Person 1 or Person 2's real code -- this document tells you exactly what fake/mock data to build against, matching the exact shape their real components will eventually produce. Swap in the real connection only in Phase 3, and only if it's ready; otherwise present against your mock data and merge later.

Your Role

You build every screen a human looks at: the caller-side call simulation, the agent's live risk console, the case timeline, and the administrator's aggregate dashboard.

The Shared Contract (Build Your Mock Data to This Exact Shape)

```
{
  "type": "transcript_update",
  "text": "string",
  "risk_vector": {
    "scores": {"acute_distress": 0.0, "depression": 0.0,
"self_harm_risk": 0.0, "fear_of_retaliation": 0.0, "intimidation":
0.0, "dissociation": 0.0, "social_isolation": 0.0,
"chronic_trauma_indicators": 0.0},
    "confidence": {"acute_distress": 0.0},
    "matched_evidence": {"acute_distress": ["example phrase"]}
  },
  "svi": {"value": 0.0, "bucket": "low | moderate | high |
critical"},
  "silence_events": [{"start_sec": 0.0, "duration_sec": 0.0,
"placement": "mid_sentence"}]
}

{"type": "action_update", "action": {"action":
"police_intervention", "state": "awaiting_senior",
"requires_senior": true}}
```

Phase 1 -- Build a Mock Data

Generator and Every Screen Shell

First, write a small JavaScript mock-data function that returns fake messages matching the exact shapes above, with values that change every 2 seconds (simulate an escalating call: SVI climbing from 20 to 85 over ten fake updates). Use this to drive every UI component you build in this entire document -- you do not need any real backend running at any point until Phase 3.

Build these screens as React components, each rendering from your mock data generator:

1. **Caller-facing IVR-sim UI** -- a phone-style interface: header text "NHAA 14566 -- incoming", a dropdown for language (Hindi / English / Hinglish), a green "Receive call" button, a call timer that counts up, a placeholder waveform (a simple animated CSS bar chart is enough), and a pre-call consent screen with three checkboxes labeled exactly: "I agree to recording", "I agree the AI can look at it", "I agree to help improve the system later (you can say no to this one)", plus buttons "Decline all" and "Accept selected".
2. **Agent console** -- a live transcript text area that appends new text every mock update, eight labeled horizontal progress bars (one per dimension, label them exactly: Acute Distress, Depression, Self-Harm Risk, Fear of Retaliation, Intimidation, Dissociation, Social Isolation, Chronic Trauma), a large badge showing the SVI number colored green (0-25) / yellow (26-50) / orange (51-75) / red (76-100), and a suggested-actions panel with five rows (Counseling, Legal Aid, Police Intervention, Witness Protection, Emergency Escalation), the last three visually greyed out with the text "REQUIRES SENIOR APPROVAL" until their mock state says otherwise.

Phase 2 -- Explainability, Confidence, Timeline, and Alerts

Add to the agent console, still driven by your mock data: - A "why" panel: for each dimension with a score above 0, display the text from `matched_evidence` next to that dimension's bar. - A confidence indicator per dimension: a single percentage number and a color (green above 80%, yellow 50-80%, red below 50%) -- deliberately simple, not a chart or graph. - A separate, larger, system-wide "AI Confidence" indicator, positioned above the eight dimension bars, not next to any single one of them -- this reflects the overall `confidence` value the real pipeline will eventually produce, not any one dimension's confidence. When this value (in your mock data, make it adjustable for testing) drops below 50%, display the text "Low confidence -- this recommendation should not be used for high-stakes action alone" directly beneath it in red. - A case timeline: a scrolling list of text lines like "00:12 -- Distress detected", generated from your mock data's timestamps and score changes. - An alert banner that appears when your mock data includes a `silence_events` entry with `placement: "mid_sentence"` and `duration_sec` over 2, reading "Unusual mid-narrative silence detected". - A second, visually distinct alert banner

for a mock "Silent SOS triggered" event (add a button to your mock-data generator that lets you manually trigger this for testing, since no other component exists yet to trigger it for real).

Phase 3 -- Real Backend Connection (Only Once Real Components Exist)

Replace your mock-data generator with a real WebSocket connection to whatever backend URL is provided at merge time. Everything you built in Phase 1 and 2 should work unchanged, since you built strictly against the shared contract shape. If the real connection produces anything that doesn't match your mock shape exactly, that is a bug in the other person's component, not yours -- flag it rather than changing your own code to work around it.

Phase 4 -- Administrator Dashboard and Accessibility Bridge

Admin dashboard -- build a separate screen showing: total case count, a bar chart of risk-bucket distribution (Low/Moderate/High/Critical counts), a bar chart of language distribution, referral-volume counts, and a **Supervisor Review Queue** panel -- a simple table listing case IDs flagged for review (driven by mock data for now: a JSON list of fake case IDs with a bucket label each), with a "Mark reviewed" button next to each row. Drive this from a mock aggregate-data generator (a simple JSON object with these numbers, changeable in your code for testing). Add one specific rule: if any single displayed count represents fewer than 5 cases, render the text "insufficient data" in that spot instead of the real number -- build this rule now, using your mock data to test it, since it is a specific requirement from the solution document's privacy commitments.

Accessibility bridge UI -- build a simple text-only chat interface (a message list plus a text input box) that renders the identical risk-panel, SVI-badge, and action-panel components you already built for the voice-call agent console, reused as-is, proving a text-channel caller gets the same visual treatment as a voice caller.

Phase 5 -- Agent-Wellbeing Panel and Package for Merge

Build a small, separate supervisor-only panel showing a mock "Agent Wellbeing" indicator: a simple gauge or percentage labeled clearly "Wellbeing Signal (not a performance score)", driven by mock data for now. Then package your entire frontend as a single buildable app (npm run build) and write a README_interfaces.md listing every component you built, the exact WebSocket URL variable that needs to be set to

connect to the real backend at merge time, and a short note on which parts are still running against mock data if the real connection isn't ready yet.